Home Exam 1

IN4230 Nettverk

Candidate number

15169

Routing protocol

The implemented routing protocol is a Distance Vector Routing (DVR) protocol built as an extension to a MIP daemon. The routing daemon maintains a routing table with information about reachable destinations, their corresponding next hop, and the cost, number of hops. Routes are exchanged periodically through UPDATE messages and neighbor discovery is handled with HELLO messages. The protocol also supports Poison Reverse to avoid count-to-infinity loops.

Periodic updates

- Every fixed interval, the node broadcasts its routing table to all direct neighbors using UPDATE messages. The UPDATE message contains the node's entire routing table, where each entry includes a destination address and its cost.
- For each neighbor, the Poisoned Reverse rule is applied: any route that was learned via that same neighbor is advertised with a cost of 255 (infinity).
- This ensures that a neighbor never believes it can reach a destination through the node that originally learned the route from it

Route Learning and Updates

- When a node receives an UPDATE message, it processes all pairs of destination and cost values from the payload
- For each entry (dest, cost), the node computes a potential new cost as advertised_cost + 1, representing the additional hop distance to the sending neighbor
- If the new path has a shorter cost, or the destination is previously unknown, or if the route was previously learned through the same neighbor, the routing table is updated with:
  - dest = destination
  - next_hop = senders mip address
  - cost = new_cost
- Routes with a cost of 255 are treated as unreachable and ignored

Neighbor Discovery

- Nodes periodically send HELLO broadcasts to detect direct neighbors.
- Upon receiving a HELLO message, the sender's MIP address is either added to the neighbor table or the timestamp is refreshed to indicate that the neighbor is still active

Link Failure and Recovery

- Each node continuously checks whether it has heard from its neighbors within a defined timeout interval.
- If a neighbor fails to send HELLO or UPDATE messages within this time, the neighbor is marked as inactive, and all routes that use that neighbor as next hop are invalidated and removed from the routing table.
- When routes are removed, a triggered update is sent immediately, ensuring that other nodes quickly learn that the affected destinations are no longer reachable.
- If the failed link later recovers, the node will again detect the neighbor through HELLO messages, rebuild the direct route, and the network will converge via periodic updates

Message format

All routing messages are exchanged as payloads encapsulated within MIP SDUs using the SDU_TYPE_ROUTING (0x04).

The routing daemon defines internal message types to distinguish between different kinds of routing messages.RT_MSG_HELLO (0x01), RT_MSG_UPDATE (0x02).

The internal message types are included inside the payload of the MIP message.
This means that the MIP daemon only handles messages of the general routing type (0x04), while the routing daemon itself interprets the specific message type based on its own format.

Hello message

Used for neighbor discovery. Sent periodically as a broadcast (TTL = 1).

The payload is in byte index 0 and contains the sdu type RT_MSG_HELLO (0x01)

Field	Bytes	Example
Destination	1	255 (broadcast)
TTL	1	1
Payload	1	0x01 (HELLO)

Update message

Advertises known routes to neighbors (Distance Vector), sent periodically.
Contains pairs of <destination, cost> entries for all valid routes in the routing table

The payload cotains the sdu type in index 0, RT_MSG_UPDATE (0x02) and the rest of the payload is the routing table entries, where there is 2 bytes per route [dest_1, cost_1, dest_2, cost_2, dest_3, ...].

Field	Bytes	Example
Destination	1	Neighbor MIP address
TTL	1	1
Payload	1 + 2xN	[0x02, dest_1, cost_1,]

Request message

Sent by the MIP daemon to routing deamon when it needs to know the next hop for a given destination.

The message is in a fixed 6-byte format, specified by the assignment:
{ my_addr, 0, 'R', 'E', 'Q', dest }

Response message

Sent by Routing deamon back to MIP deamon in response to a REQ message.
Informs MIPD which neighbor to forward the packet to.

The message is in a fixed 6-byte format, specified by the assignment:
{ my_addr, 0, 'R', 'S', 'P', next }

Count-to-Infinity problem in DVR protocols

In Distance Vector Routing (DVR) protocols, the Count-to-Infinity problem occurs when routers keep updating each other with increasing path costs to a destination that has become unreachable. The problem occurs when routers dont realize that a route they learn from a neighbor actually loops back through themselves.

This will happen because each node bases its routing decisions on information received from its neighbors. So if one neighbor continues to send route updates on a failed destination, other routing nodes may continue to increase the distance step by step, making them "count to infinity" before realizing that the route is no longer valid.

Example:

1. Node A reaches destination Z through node B.
2. When the link between B and Z fails, B removes its direct route but still receives an update from A, that says “I can reach Z.”
3. Because B does not know that A originally learned that route from B, it assumes A has found another path and updates its own route to “Z via A.”
4. Both nodes now point to each other for the same destination, and with every new update they increase the cost

My implementation handles the Count-to-Infinity problem using two main mechanisms: Poison Reverse and route invalidation with triggered updates**.**

Poison Reverse

- When a node sends its routing table to its neighbors, it advertises any route that was originally learned through that same neighbor with a cost of 255 (infinity)**.**
- This prevents the neighbor from thinking that the route can be reached through the same node, which will break potential loops.

Route Invalidation and Triggered Updates

- Each node controlls the time since it last received HELLO or UPDATE messages from its neighbors.
- If a neighbor stops responding within a timeout interval, all routes that use that neighbor as the next hop are invalidated and set to infinity.

- When routes are removed, the node sends a triggered broadcast update to inform the rest of the network about the change

Limitation of Split Horizon and How Poison Reverse Compares

Split Horizon prevents routing loops by not advertising routes back to the neighbor they were originally learned from. However, this approach can also create uncertainty between routers, since it just omits the routes instead of explicitly marking them as unreachable. The lack of clarity can lead to misunderstandings about which paths are still valid during network changes.

Poison Reverse builds upon Split Horizon by making this rule explicit. Instead of staying silent, it tells the neighbor that any route learned through them is unreachable by advertising it with an infinite cost. This more explicit communication reduces the risk of misunderstandings, prevents unnecessary route suppression, and makes routing behavior more predictable.

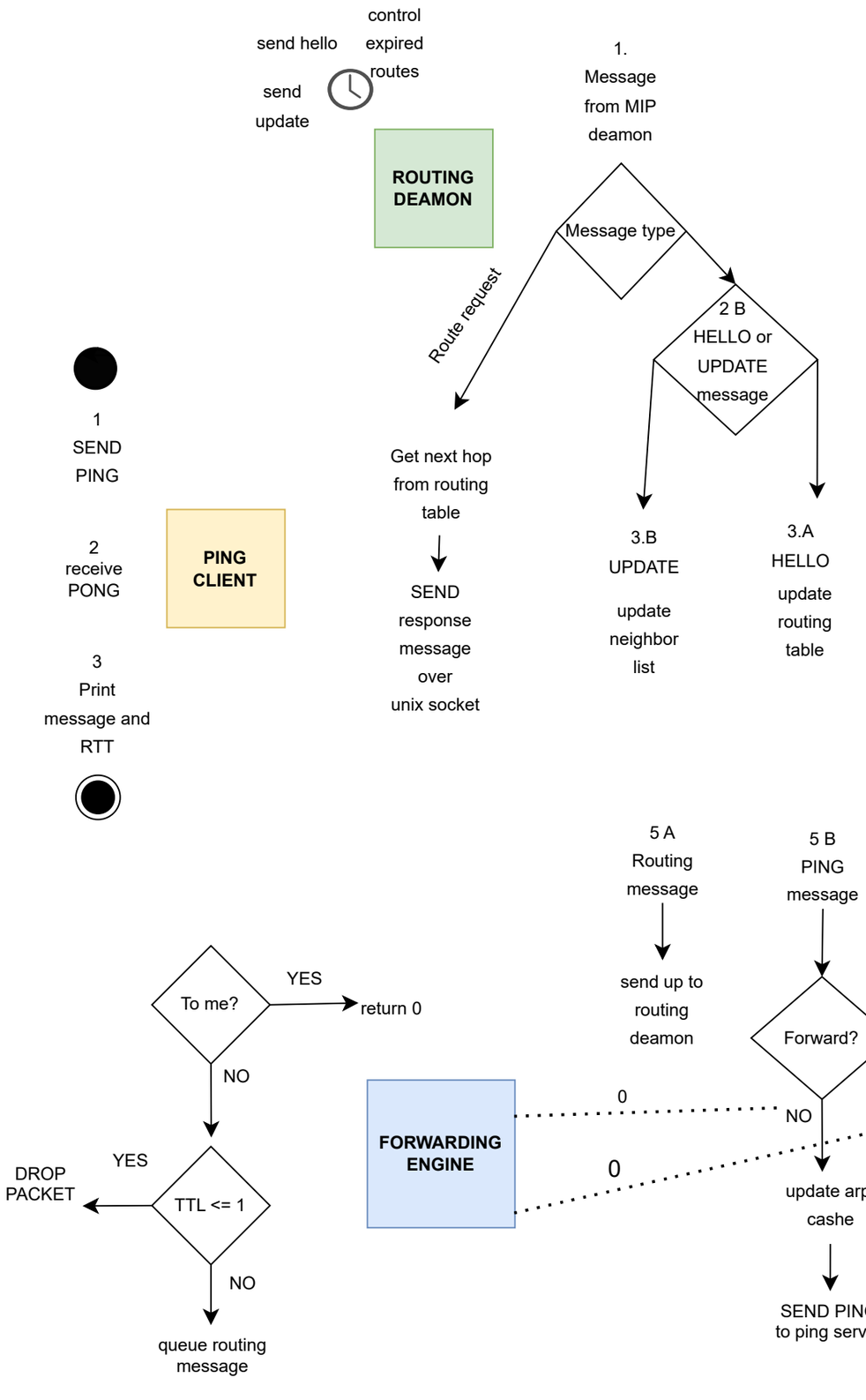
Furthermore, while Split Horizon can prevent direct two-node loops, it does not necessarily stop indirect loops. In situations where routing information travels through several intermediate routers and then returns to its original source through a different path, a loop can still occur. This happens because the router receiving the update does not realize that the route ultimately points back to itself.

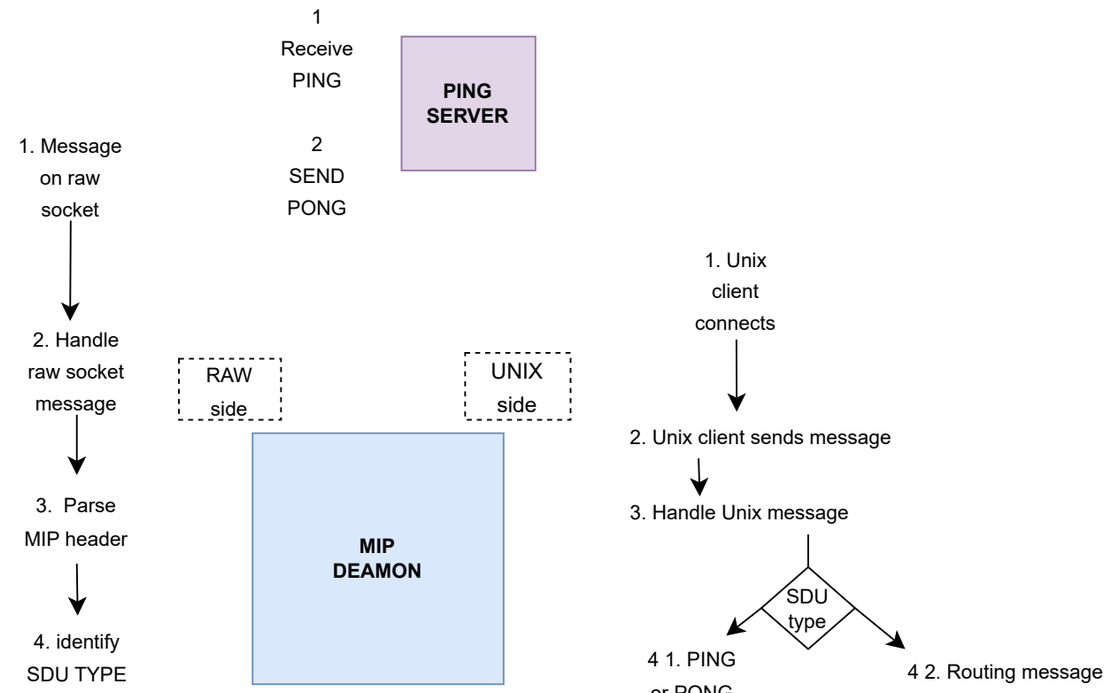
Without Poison Reverse, such information can circulate indirectly between nodes and continue to increase in cost with each update. For example, if node C advertises that it can reach destination X, nodes A and B may learn this route through C and then start advertising it to each other. Since Split Horizon only blocks updates from being sent back to the same neighbor, A can still tell B about X, and B may later tell C, creating an indirect loop.

With Poison Reverse, A and B explicitly tell C that X is unreachable through them by advertising the route with an infinite cost. They do this because both A and B are direct neighbors of C and originally learned the route to X through it. According to the Poison Reverse rule, any route learned from a neighbor is advertised back to that same neighbor with an infinite cost, preventing that neighbor from ever routing traffic through them for that destination. This makes C aware that no valid alternative path exists through its neighbors and therefore prevents the loop.

Notes on use of AI

I used AI as a support tool to help me understand C programming as i did not have previous experiance with the language before taking this course. I also used it to debug errors, and improve the structure of my code. In some cases, I also received suggestions for code snippets, which I adapted and integrated myself after verifying them against the assignment requirements.


send hello
control expired routes
send update
ROUTING DEAMON
1. Message from MIP deamon
Message type
Route request
2 B HELLO or UPDATE message
Get next hop from routing table
SEND response message over unix socket
3.B UPDATE
update neighbor list
3.A HELLO
update routing table
1 SEND PING
2 receive PONG
PING CLIENT
3 Print message and RTT

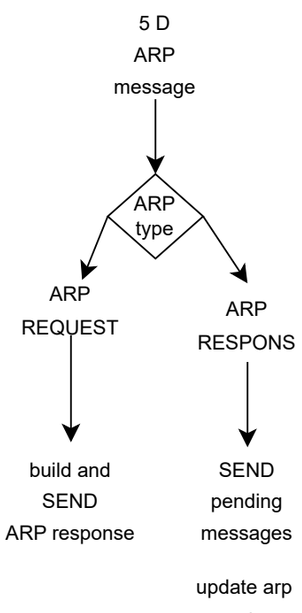

1 Receive PING
2 SEND PONG
PING SERVER
1. Message on raw socket
2. Handle raw socket message
3. Parse MIP header
4. identify SDU TYPE
RAW side
UNIX side
MIP DEAMON

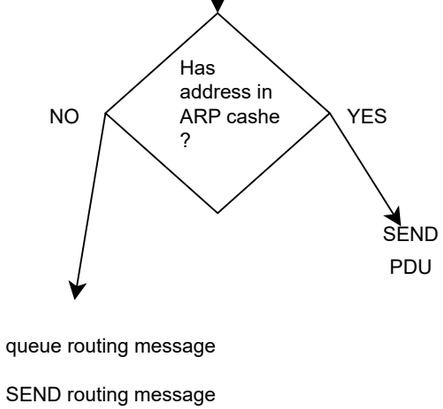
1. Unix client connects
2. Unix client sends message
3. Handle Unix message
SDU type
4 1. PING or PONG
4 2. Routing message

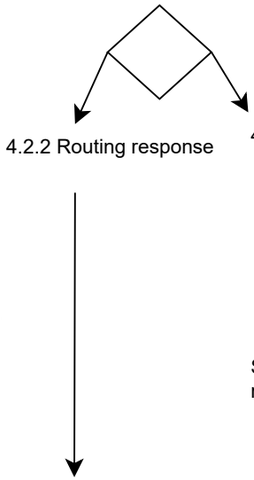
To me?
YES
return 0
NO
TTL <= 1
YES
DROP PACKET
NO
queue routing message
SEND routing request

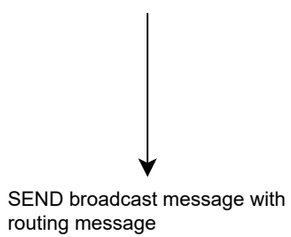
FORWARDING ENGINE

5 A Routing message
send up to routing deamon
5 B PING message
Forward?
0
NO
update arp cashe
SEND PING to ping server
5 C PONG message
Forward?
0
NO
update arp cashe
SEND PONG to ping client


5 D ARP message
ARP type
ARP REQUEST
build and SEND ARP response
ARP RESPONS
SEND pending messages
update arp cashe


Has address in ARP cashe ?
NO
YES
queue routing message
SEND routing message
SEND PDU


4.2.2 Routing response
SEND first valid packet in routing queue


4.2.1.HELLO or UPDATE
SEND broadcast message with routing message